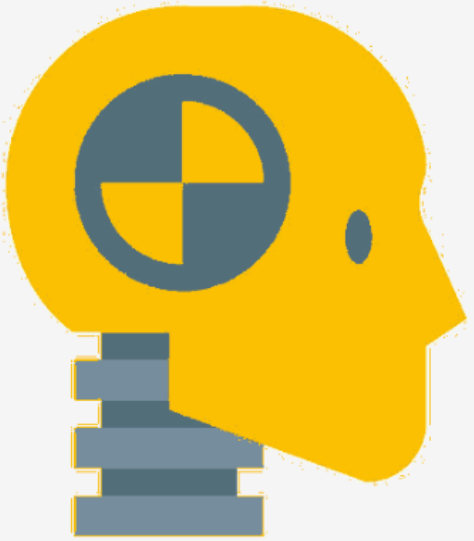
BUCKLE UP

Claude Code Crash Course

Learn the fundamentals. Everything else is a plugin.



Kamau Washington Portfolio CTO, Mphasis

BEFORE WE START

Any of them will do

Claude Code is the worked example here. Everything in this deck is a fundamental of driving a frontier model. Swap the name and the concepts still hold.

Another vendor

Same picker, same effort dial, a different logo on the invoice.

An assisted IDE

Whatever your team already pays for. The loop underneath is the loop.

Work that is not code

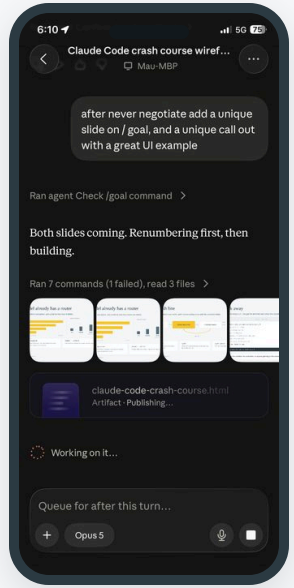
Research, contracts, runbooks, spreadsheets. Same context, same tools, same hooks.

> Nobody paid me to say Claude. It is just the one that was already open at 6:10 in the morning. 😉

WHO IS TALKING

This guy

ROLE	Portfolio CTO at Mphasis
ARCHITECT	25 years. Application, Solution, Enterprise.
HANDS ON	Still on the keyboard, on purpose
PASSION PROJECT	BOE, Bag of Engines. boerules.com
PLAYS	Classical guitar and piano



6:10am, Starbucks line, editing this deck from my phone.

> “ Solving problems is challenging, and often rewarding. Coding is just tedious after 25 years.

WHAT A TIME TO BE HERE

> “You all used to hand write a million lines of code?”
>
> Somebody, ten years from now, opening your repo

We are the 2026 equivalent of COBOL programmers.

I HAVE SEEN THIS MOVIE

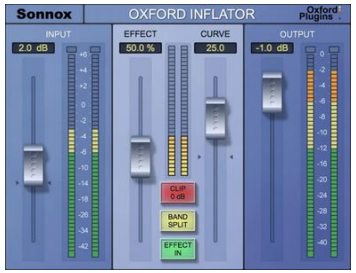
The plugin pileup

The chain everybody swore by. Six plugins deep before anyone asked how the snare was recorded.

RVox
VOCALS

Crunchessor
SNARE

C4
CYMBALS



Inflator
GLUE

L3
LAST IN THE CHAIN

So I learned compression, EQ, gain staging, limiting, ducking, side chaining. *The basics.*

After that, a solid mix in any room with whatever shipped in the Digital Audio Workstation or Analog Console that served as Main.

THE WHOLE POINT

Learn the fundamentals. Tune out the noise.

DAY ONE SETUP, APPARENTLY

Six frontier models. A pile of open source models. A router you either hand rolled or bought.

A fine hobby. Come back to it later.

WHAT ACTUALLY WORKS

One frontier product. The models that ship with it. The fundamentals underneath.

Phenomenal results immediately.

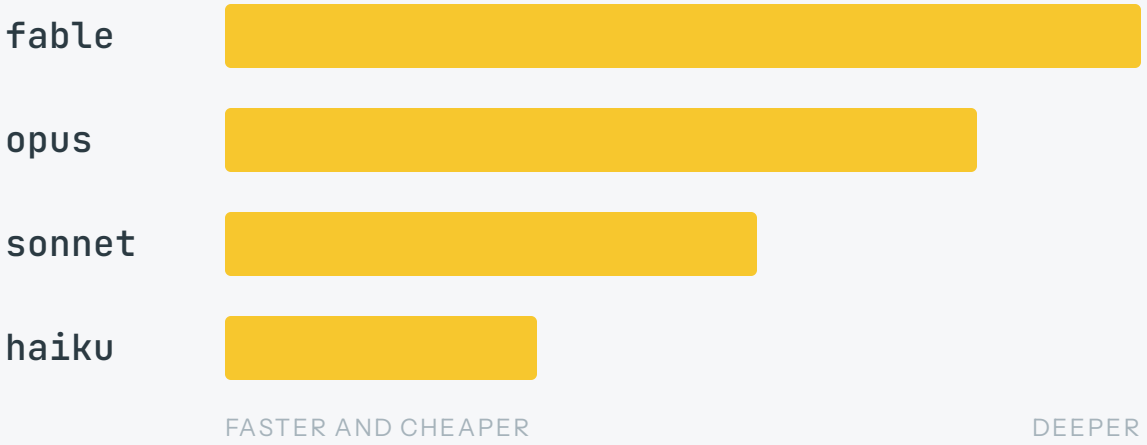
The noise will still be there when you get back.

FUNDAMENTALS

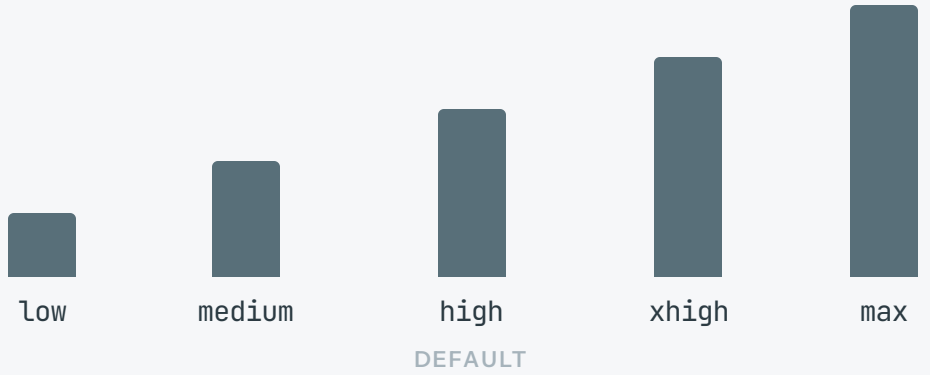
Your model already has a router

Four models behind one picker, and a dial for how hard each one thinks.

THE MODELS



THE EFFORT



`default · best · opusplan`

Claude routes for you. default follows your plan, best takes the newest, opusplan plans with Opus and executes with Sonnet.

`/model · /effort`

Change either mid session. Agents can carry their own model and effort.

A WORD OF CAUTION

“Just because you watch Bruce Lee movies doesn’t mean you know kung fu.”

The demo lands in an afternoon. Somebody has to run it for the next four years.

“

I can rebuild Salesforce.

IN HOUSE, OVER A LONG WEEKEND

“

I don’t need SaaS.

WE CAN JUST WRITE THAT

“

Spring Boot to Sanic.

NOBODY HERE WRITES PYTHON

WHAT TO WEIGH

- The next team
- Your team makeup and capability
- The operational model of the thing

Point it at learning the thing you are about to own. One shot to prod is still a demo, and demos do not carry a pager.

Commands

Anything that steers the session rather than the code starts with a slash.

/clear

Wipes the thread. Fresh start, same folder.

/compact

Squeezes the history into a summary. Same thread, smaller.

/context

Shows what is eating the window right now.

/usage

Shows what you burned and what is left.

custom

A markdown file in .claude/commands becomes /yours.

Four commands and a folder. That is most of context management.

Skills

A folder of instructions for one job, loaded when that job shows up.

loaded

Claude infers it from the ask, or you call it by name.

named

The name is the trigger. Name it for the job.

borrowed

Runs on the model and judgment of the agent that loads it.

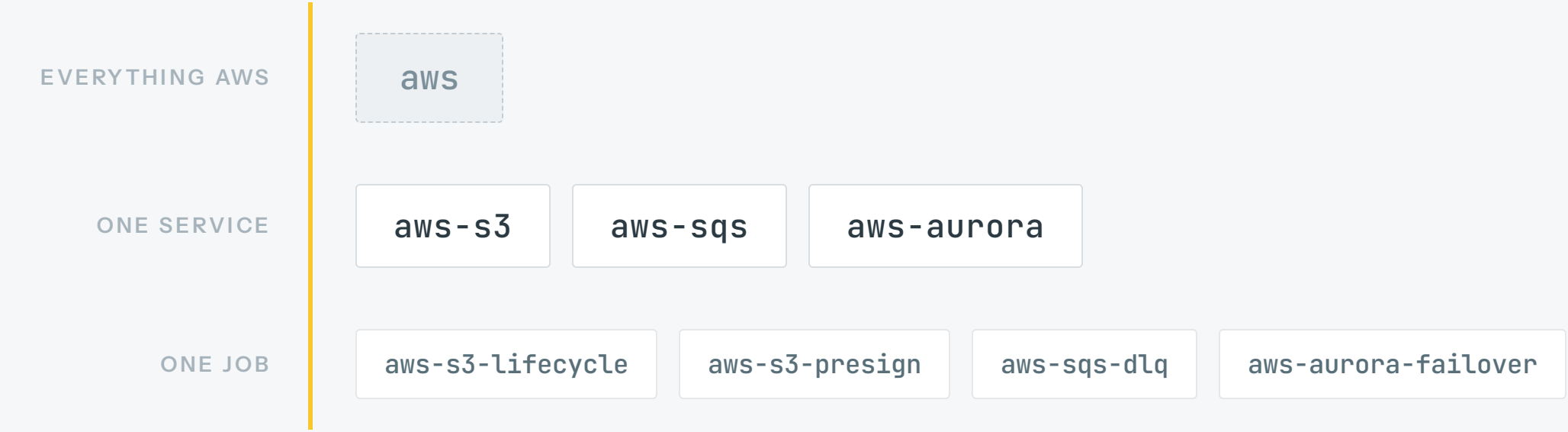
scoped

One skill, one job. Granularity wins.

You are writing for something already smart. Say the job, then move.

Small enough to aim

A skill named for a whole platform never gets picked with any confidence. Keep splitting.



> Keep splitting until the name alone tells you when to reach for it.

Agents

Work you hand off runs somewhere else, on its own context, under a name you gave it.

named

A name and a personality. Mine are Kill Bill characters.

tuned

Model and effort, set per agent.

light

Keep your orchestrator light. It routes, it does not read.

handed off

One finishes and passes the work to the next.

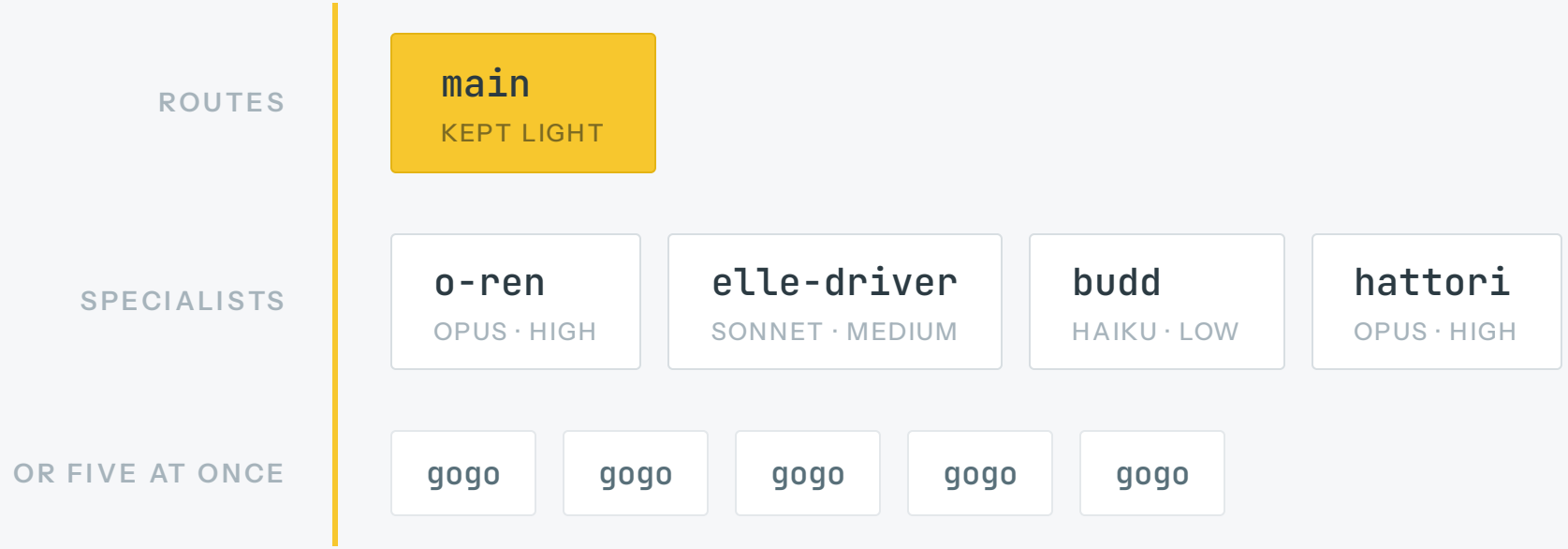
cloned

Run five of the same agent on five slices at once.

Each one gets a clean context window. That is the whole trick.

Name the bench

Main routes. The named agents read, each on its own model and its own context.



> Names you actually remember turn delegation into a reflex. Mine came from a movie.

FUNDAMENTALS

The backlog is the memory

Project memory bloats and drifts on every machine. The issues are the source of truth. GitHub, Jira, whatever you already run.

THE RITUAL

Starting a project

Hours on a one pager. It ends up twenty pages. Every idea, every gotcha, the architecture, the service level objectives, the goals.

Starting a day

An hour reading where you stopped and writing the next stretch. Or skip it and take the next issue when the backlog already runs deep.

WHY IT HOLDS UP

Source of truth
One place. Same for everyone.

Comments are the log
Decisions, dead ends, what ran.

Anyone picks it up
Your teammate, or you Monday.

Watch the agents
Progress lands in the thread.

Memory stays local
Never commit it on a team.

> Build the wall of issues in detail up front. Then pull one down when you want a quick win and the endorphins that come with it.

Tools

Where a probabilistic model reaches out and does something deterministic.

local

A function on your machine. A script, a CLI, a bash command.

mcp

A server that hands you someone else's system. Jira, GitHub, a database.

deterministic

The model chooses. The tool runs the same way every time.

permissioned

It does what you allowed and stops there.

described

The description is what gets it picked. Same rule as skills.

The model guesses well. The tool call runs exactly. Keep the tool narrow and the result stays boring.

Where the guessing stops

The model picks the tool and fills in the arguments. Everything after that is ordinary code.

	LOCAL FUNCTION	MCP SERVER
THE ASK	Add a retry to message post	File that as a bug
THE CALL	`Write("src/msgs-api.ts", …)`	`POST /rest/api/3/issue`
WHAT CHANGED	Retry added to the POST method	PROJ-412 created in Jira

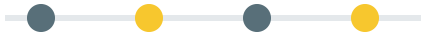
Hooks

A shell command that fires on an event, with no model in the loop.

deterministic

A shell command on an event. The model gets no vote.

events



PreToolUse, PostToolUse, UserPromptSubmit, SessionStart, Stop.

blocking

Exit 2 on PreToolUse stops the call and hands Claude the reason.

configured

settings.json, matched on the tool name.

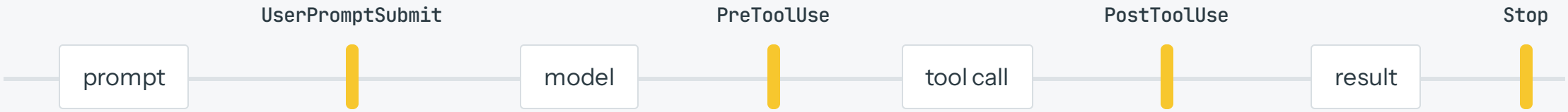
what for

Format on write. Block the dangerous command. Re-inject your rules.

Anything you refuse to negotiate belongs in a hook.

Fixed points in the loop

The loop stops at the same four places every run. You decide what runs there.



.CLAUDE/SETTINGS.JSON

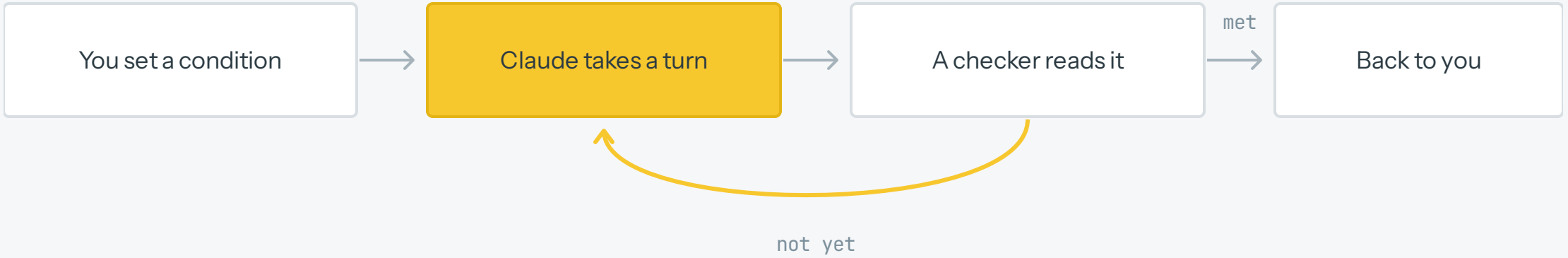
```
"hooks": {
  "PostToolUse": [
    { "matcher": "Edit|Write",
      "hooks": [{ "type": "command",
                  "command": "./verify.sh" }] }
  ]
}
```

WHAT THAT BUYS YOU

verify.sh formats the file and runs the type check. Whatever fails goes straight back to Claude, so it fixes its own mistakes before you ever read the diff.

Set a finish line

Give Claude a condition it can verify, and it keeps taking turns until the condition holds.



`/goal <condition>`

Write it so a script could check it. Exit codes, test runs, a build that passes.

`/goal`

On its own it reports the turns spent and what the checker said last.

`/goal pause · clear`

Take the wheel back whenever you want it.

Then walk away

The checker runs after every turn. You get the terminal back when the condition is true.

claude — msgs-api

```
> /goal all unit tests pass and npm run build exits 0
● Goal set. Working until it holds.
✓ turn 1 · edited src/msgs-api.ts
✓ turn 2 · npm test → 2 failing
● checker: build passes, 2 tests still failing
✓ turn 3 · fixed the retry backoff
● checker: all tests pass, build exits 0
✓ Goal met in 3 turns. Back to you.
```

```
● goal: all unit tests pass and npm run build exits 0          3 turns · 41k tokens
```

> The status line carries the condition the whole time, so anyone glancing at the screen knows what done means.

ENOUGH SLIDES

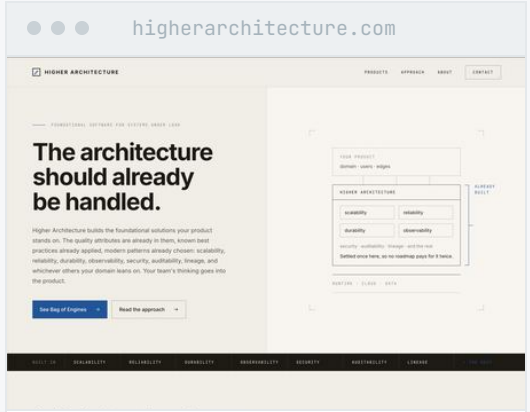
Demo

Live, unrehearsed, and hopefully upright at the end of it.

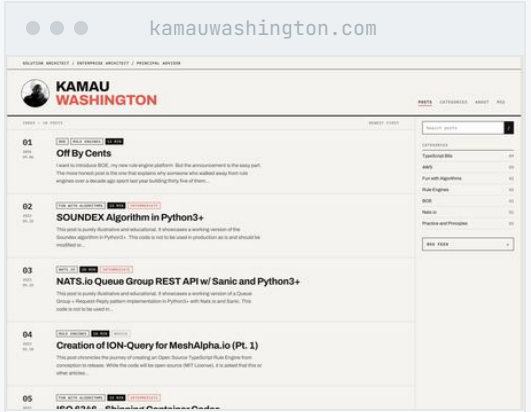
Thank you!

Designed, built, and operated on exactly the fundamentals in this deck, by one person.

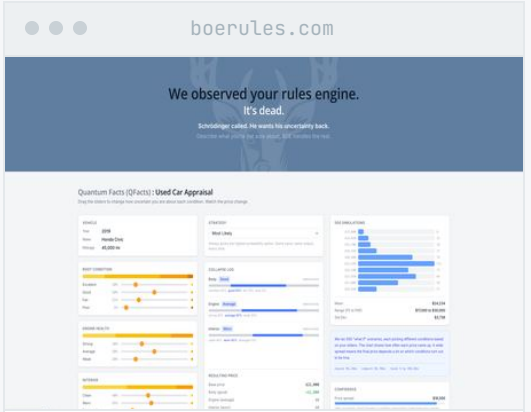
KAMAU'S PERSONAL WORK



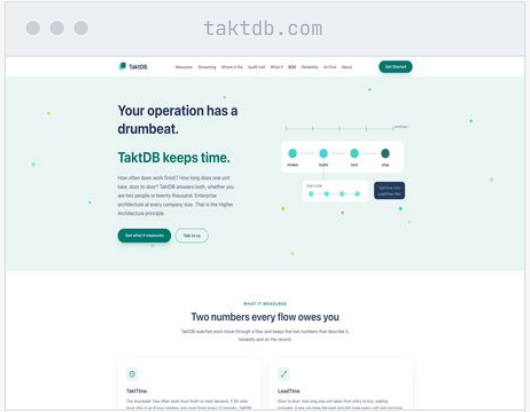
`higherarchitecture.com`



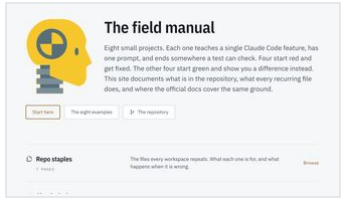
`kamauwashington.com`



`boerules.com`



`taktdb.com`



`cccc.kamauwashington.com`

This deck, the field manual, and the repo behind it. Take the whole thing.